МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №7

Специальность ПО-13

Выполнил:
М.А.Шумило
студент группы
ПО-13
Проверил:
А.Д.Кулик
25.03.2026

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений.

Ход работы

Задание 1. Построение графических примитивов и надписей
Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

9) Создать классы Point и Line. Объявить список из n объектов класса Point. Для объекта класса Line определить, какие из объектов Point лежат на одной стороне от прямой линии и какие на другой. Реализовать ввод данных для объекта Line и случайное задание данных для объекта Point.

Выполнение:

Код программы:

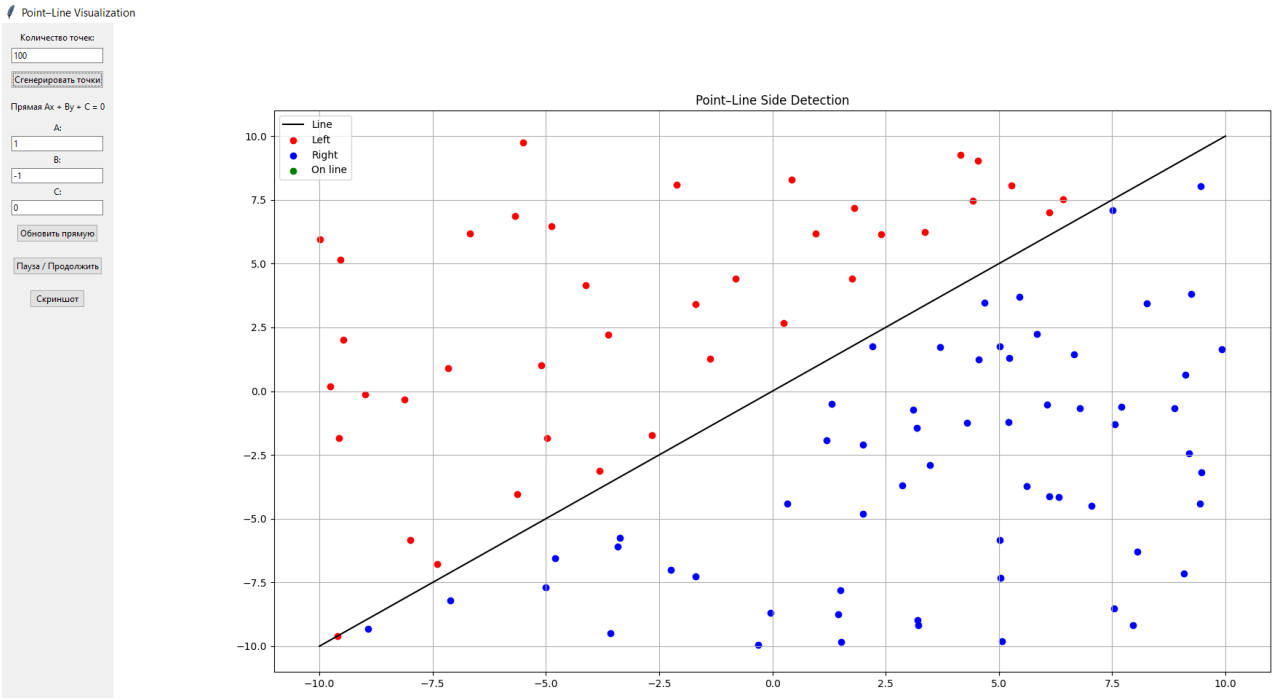
```
import tkinter as tk
from tkinter import ttk
from tkinter import messagebox
import random
import matplotlib
matplotlib.use("TkAgg")
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt


class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class Line:
    def __init__(self, A, B, C):
        self.A = A
        self.B = B
        self.C = C

    def side(self, p: Point):
        return self.A * p.x + self.B * p.y + self.C

class App:
    def __init__(self, root):
        self.root = root
        self.root.title("Point-Line Visualization")

        self.points = []
        self.line = Line(1, -1, 0)
        self.paused = False
        self.after_id = None  # ID таймера

        # UI
        self.build_ui()

        self.fig, self.ax = plt.subplots(figsize=(6, 6))
        self.canvas = FigureCanvasTkAgg(self.fig, master=self.plot_frame)
```

```
        self.canvas.get_tk_widget().pack(fill="both", expand=True)

        # Обработчик закрытия окна
        self.root.protocol("WM_DELETE_WINDOW", self.on_close)

        # Запуск цикла обновления
        self.update_plot()

    def build_ui(self):
        control = ttk.Frame(self.root)
        control.pack(side="left", fill="y", padx=10, pady=10)

        ttk.Label(control, text="Количество точек:").pack()
        self.n_entry = ttk.Entry(control)
        self.n_entry.insert(0, "20")
        self.n_entry.pack(pady=5)

        ttk.Button(control, text="Сгенерировать точки",
command=self.generate_points).pack(pady=5)

        ttk.Label(control, text="Прямая Ax + By + C = 0").pack(pady=10)

        self.A_entry = ttk.Entry(control)
        self.B_entry = ttk.Entry(control)
        self.C_entry = ttk.Entry(control)

        for label, entry, val in [
            ("A:", self.A_entry, "1"),
            ("B:", self.B_entry, "-1"),
            ("C:", self.C_entry, "0")
        ]:
            ttk.Label(control, text=label).pack()
            entry.insert(0, val)
            entry.pack(pady=2)

        ttk.Button(control, text="Обновить прямую",
command=self.update_line).pack(pady=10)

        ttk.Button(control, text="Пауза / Продолжить",
command=self.toggle_pause).pack(pady=10)
        ttk.Button(control, text="Скриншот",
command=self.save_screenshot).pack(pady=10)

        self.plot_frame = ttk.Frame(self.root)
        self.plot_frame.pack(side="right", fill="both", expand=True)

    # ДЕЙСТВИЯ

    def generate_points(self):
        try:
            n = int(self.n_entry.get())
        except:
            messagebox.showerror("Ошибка", "Введите корректное число точек")
            return

        self.points = [
            Point(random.uniform(-10, 10), random.uniform(-10, 10))
            for _ in range(n)
        ]

    def update_line(self):
        try:
            A = float(self.A_entry.get())
            B = float(self.B_entry.get())
```

```
            C = float(self.C_entry.get())
        except:
            messagebox.showerror("Ошибка", "Введите корректные коэффициенты")
            return

        self.line = Line(A, B, C)

    def toggle_pause(self):
        self.paused = not self.paused

    def save_screenshot(self):
        self.fig.savefig("screenshot.png")
        messagebox.showinfo("Готово", "Скриншот сохранён как screenshot.png")

    # ВИЗУАЛИЗАЦИЯ

    def update_plot(self):
        # Если окно закрыто — не продолжаем
        if not self.root.winfo_exists():
            return

        if not self.paused:
            self.ax.clear()
            self.ax.grid(True)
            self.ax.set_title("Point-Line Side Detection")

            # Рисуем прямую
            A, B, C = self.line.A, self.line.B, self.line.C
            x_vals = [-10, 10]

            if B != 0:
                y_vals = [(-A * x - C) / B for x in x_vals]
            else:
                x_vals = [-C / A] * 2
                y_vals = [-10, 10]

            self.ax.plot(x_vals, y_vals, 'k-', label="Line")

            # Разделяем точки
            left, right, on_line = [], [], []

            for p in self.points:
                s = self.line.side(p)
                if s > 0:
                    right.append(p)
                elif s < 0:
                    left.append(p)
                else:
                    on_line.append(p)

            self.ax.scatter([p.x for p in left], [p.y for p in left],
color='red', label='Left')
            self.ax.scatter([p.x for p in right], [p.y for p in right],
color='blue', label='Right')
            self.ax.scatter([p.x for p in on_line], [p.y for p in on_line],
color='green', label='On line')

            self.ax.legend()
            self.canvas.draw()

        # Планируем следующее обновление
        self.after_id = self.root.after(200, self.update_plot)

    #ЗАКРЫТИЕ ОКНА
```

```
    def on_close(self):
        if self.after_id is not None:
            try:
                self.root.after_cancel(self.after_id)
            except:
                pass
        self.root.destroy()

#ЗАПУСК

root = tk.Tk()
app = App(root)
root.mainloop()
```

Рисунки с результатами работы программы



Задание 2. Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала.

9) Остров Минковского

Выполнение:

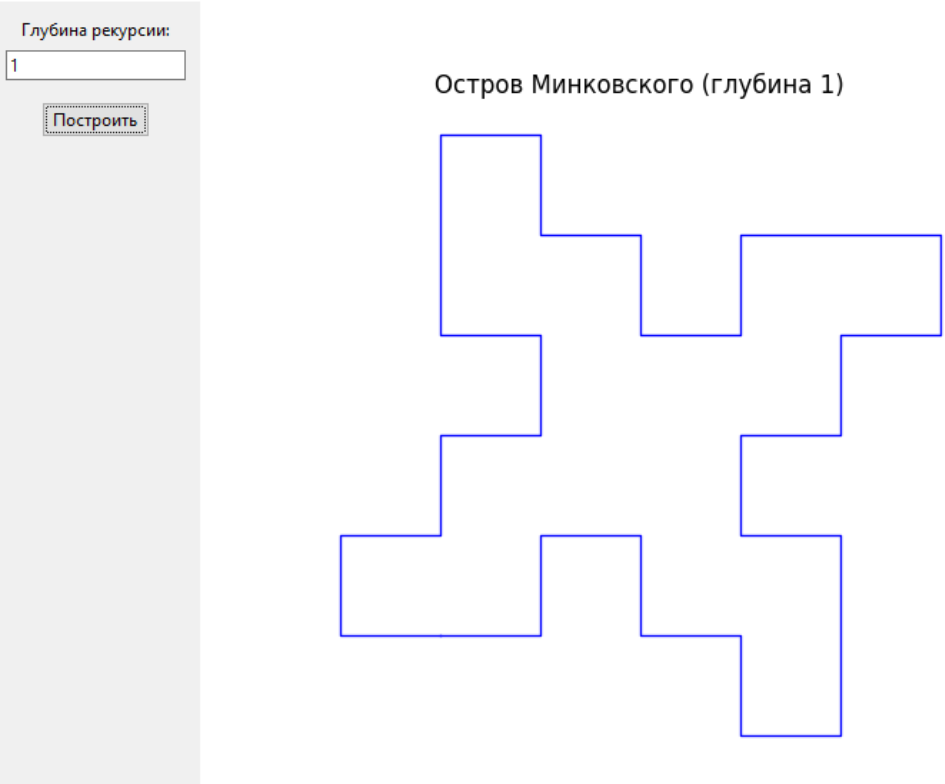
Код программы:

```
import tkinter as tk
from tkinter import ttk
import matplotlib
matplotlib.use("TkAgg")
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt

# ФРАКТАЛ
def minkowski_curve(x1, y1, x2, y2, depth, points):
    if depth == 0:
        points.append((x1, y1))
        return
```

```
    dx = (x2 - x1) / 4
    dy = (y2 - y1) / 4

    x = x1
    y = y1

    pts = [
        (x, y),
        (x + dx, y + dy),
        (x + dx - dy, y + dy + dx),
        (x + 2*dx - dy, y + 2*dy + dx),
        (x + 2*dx, y + 2*dy),
        (x + 3*dx, y + 3*dy),
        (x + 3*dx + dy, y + 3*dy - dx),
        (x + 4*dx + dy, y + 4*dy - dx),
        (x2, y2),
    ]

    for i in range(len(pts) - 1):
        minkowski_curve(pts[i][0], pts[i][1],
                        pts[i+1][0], pts[i+1][1],
                        depth - 1, points)

def minkowski_island(depth):
    points = []
    minkowski_curve(0, 0, 1, 0, depth, points)
    minkowski_curve(1, 0, 1, 1, depth, points)
    minkowski_curve(1, 1, 0, 1, depth, points)
    minkowski_curve(0, 1, 0, 0, depth, points)
    points.append((0, 0))
    return points

# ПРИЛОЖЕНИЕ

class App:
    def __init__(self, root):
        self.root = root
        self.root.title("Фрактал: Остров Минковского")

        self.depth = 1

        self.build_ui()

        self.fig, self.ax = plt.subplots(figsize=(6, 6))
        self.canvas = FigureCanvasTkAgg(self.fig, master=self.plot_frame)
        self.canvas.get_tk_widget().pack(fill="both", expand=True)
        self.root.protocol("WM_DELETE_WINDOW", self.on_close)

        self.draw_fractal()

    def build_ui(self):
        control = ttk.Frame(self.root)
        control.pack(side="left", fill="y", padx=10, pady=10)

        ttk.Label(control, text="Глубина рекурсии:").pack()
        self.depth_entry = ttk.Entry(control)
        self.depth_entry.insert(0, "1")
        self.depth_entry.pack(pady=5)

        ttk.Button(control, text="Построить",
command=self.update_depth).pack(pady=10)

        self.plot_frame = ttk.Frame(self.root)
```

```
        self.plot_frame.pack(side="right", fill="both", expand=True)

    def update_depth(self):
        try:
            self.depth = int(self.depth_entry.get())
        except:
            self.depth = 1

        self.draw_fractal()

    def draw_fractal(self):
        self.ax.clear()
        pts = minkowski_island(self.depth)

        xs = [p[0] for p in pts]
        ys = [p[1] for p in pts]

        self.ax.plot(xs, ys, color="blue", linewidth=1)
        self.ax.set_aspect("equal")
        self.ax.set_title(f"Остров Минковского (глубина {self.depth})")
        self.ax.axis("off")

        self.canvas.draw()
    def on_close(self):
        plt.close(self.fig)
        self.root.destroy()

root = tk.Tk()
app = App(root)
root.mainloop()
```

Рисунки с результатами работы программы



Вывод: освоил возможности языка программирования Python в разработке оконных приложений.